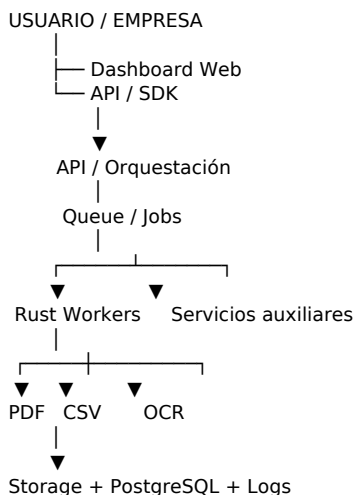


PLATAFORMA DE PROCESAMIENTO DE DOCUMENTOS Y DATOS

Documento técnico y de producto — visión 0% → 100%

Stack principal: Next.js + TypeScript + Rust + PostgreSQL + Redis + almacenamiento de objetos

Objetivo: construir una plataforma SaaS API-first para procesamiento masivo de PDF/documentos, limpieza y transformación de CSV/datos, OCR local y generación de PDF mediante API.



Documento pensado para: leer la visión completa, convertirla en backlog y entregársela a Cursor para que diseñe el plan de implementación por fases.

1. Visión del proyecto

Concepto: una plataforma de procesamiento de archivos y datos que permite subir documentos desde un dashboard o procesarlos programáticamente mediante API.

Problema: muchas empresas reciben grandes volúmenes de PDFs, facturas, reportes y CSV; hacer manualmente extracción, limpieza, conversión, generación y validación consume tiempo.

Propuesta: “sube tus archivos o llama nuestra API; nosotros procesamos el trabajo y entregamos un resultado estructurado”.

Principios

- API-first: todo lo que haga el dashboard debe apoyarse en servicios internos que también puedan exponerse por API.
- Asíncrono: los trabajos pesados se procesan con cola y workers, no bloqueando la petición HTTP.
- Escalable: añadir más workers debe aumentar capacidad sin rediseñar el sistema.
- Uso medible: cada operación consume créditos/unidades de uso.
- Seguridad por defecto: aislamiento de archivos, URLs temporales, límites, validaciones y limpieza automática.
- Rust donde aporta valor: procesamiento CPU-intensivo, workers, parsing y componentes de alto rendimiento; Next.js para producto web.

2. Alcance funcional

Módulo	MVP	Evolución
PDF / Documentos	Subida, extracción de texto, unión/división, compresión, metadatos	OCR, extracción estructurada, tablas, clasificación
CSV / Datos	Validación, duplicados, filtros, dividir/combinar, CSV to JSON	Seguridad, reglas avanzadas, perfiles de limpieza, jobs recurrentes
PDF Generator	Plantillas + datos → PDF	Editor visual, versiones, lotes, firmas/integraciones
OCR	Motor local/self-hosted para PDFs escaneados	Preprocesamiento avanzado, layouts, campos y confianza
API	API keys, jobs, resultados, usage	SDKs, webhooks, idempotencia, rate limits avanzados
Dashboard	Jobs, archivos, resultados, uso, billing	Equipos, roles, auditoría, analytics

3. Qué NO hacer al principio

- No construir una app móvil.
- No construir una app de escritorio en el MVP.
- No entrenar un modelo OCR propio desde cero.
- No intentar competir de frente con n8n, Zapier, Adobe o plataformas generalistas.
- No empezar con 30 operaciones: lanzar un conjunto pequeño, medible y rentable.

4. Experiencia web / Frontend

La aplicación principal será una web SaaS. Next.js será responsable de la experiencia de producto, autenticación/UI, páginas públicas y consumo de la API.

Mapa de navegación

DOMINIO	
/	Landing / ventas
/pricing	Precios
/docs	Documentación API
/login	Acceso
/signup	Registro
/app	
/dashboard	
/documents	
/data	
/pdf	
/api-keys	
/jobs	
/usage	
/billing	
/settings	

4.1 Dashboard

LOGO		Usuario	
Dashboard	Buenos días		
Documents			
Data	[8,421 créditos]	[73% usado]	
PDF	[12.4 GB procesados]		
API			
Usage	Actividad reciente		
Billing	✓ 500 PDFs procesados		
Settings	✓ clientes.csv limpiado		
	✗ job fallido		

4.2 Documents

Documents

[+ Upload files]

Todos Procesando Completados Error

invoice-001.pdf	✓ Completed
invoice-002.pdf	✓ Completed
invoice-003.pdf	⚙ Processing
invoice-004.pdf	✗ Failed

Procesando 7,421 / 10,000



Al seleccionar un trabajo: archivos de entrada, operación, progreso, duración, consumo, logs, errores y resultado.

4.3 Selección de operación

¿Qué quieres hacer?

- ☐ Extraer texto
- ☐ Extraer datos
- ☐ OCR
- ☐ Comprimir
- ☐ Dividir
- ☐ Combinar
- ☐ Convertir
- ☐ Generar CSV

[Procesar]

4.4 Extracción estructurada

Esquema de salida

Nombre	[customer_name]
NIT	[tax_id]
Fecha	[date]
Total	[total]

[+ Add field]

[Procesar]

El sistema debe guardar el esquema y permitir reutilizarlo en futuros jobs.

4.5 CSV / Data

Data

[Upload CSV]

- ☐ Limpiar datos
- ☐ Eliminar duplicados
- ☒ Validar datos
- ☐ Combinar CSV
- ☒ Dividir archivo
- ☒ Convertir formato

2,450,832 rows

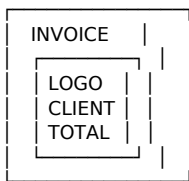
Name	Email	Phone
Juan Pérez	juan@gmail.com	300...
Pedro López	pedro@gmail.com	301...

El resultado debe mostrar un resumen: filas procesadas, duplicados, inválidos, transformaciones y archivo de salida.

4.6 PDF Generator

PDF Generator

Templates



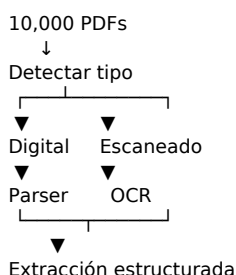
[+ Create template]

Las plantillas usan variables, por ejemplo `{{customer}}`, `{{invoice_number}}`, `{{items}}`, `{{total}}`. El mismo motor debe poder recibir datos manualmente o por API.

5. Módulos detallados

5.1 PDF / documentos

- Extraer texto de PDFs digitales sin OCR cuando exista una capa de texto.
- Detectar PDFs escaneados y enviarlos al pipeline OCR.
- Unir, dividir, comprimir y transformar PDFs.
- Extraer metadatos.
- Procesar lotes.
- Convertir resultado a texto/JSON/CSV cuando corresponda.
- Crear pipelines de extracción estructurada: texto → reglas/modelo → JSON.



5.2 CSV / datos

- Eliminar duplicados por una o varias columnas.
- Normalizar espacios, mayúsculas/minúsculas y formatos.
- Validar emails, teléfonos y tipos.
- Filtrar y ordenar.
- Combinar archivos.
- Dividir archivos grandes.
- CSV ↔ JSON.
- Procesamiento por streaming para archivos grandes, evitando cargar todo en RAM.

```

clientes.csv
2,500,000 filas
↓
Validación + normalización
↓
83,421 duplicados
12,392 emails inválidos
7,812 teléfonos inválidos
↓
clientes_limpios.csv
  
```

5.3 OCR

OCR = reconocimiento óptico de caracteres. Debe utilizarse cuando el documento es una imagen/escaneo y no contiene texto digital extraíble.

```

PDF escaneado
↓
Renderizar página
↓
Preprocesamiento
↓
Motor OCR self-hosted
↓
Texto + coordenadas/confianza
↓
Parser / extracción
↓
JSON estructurado
  
```

Decisión técnica: Rust puede ser el servicio/orquestador del pipeline y ejecutar un motor OCR local/open source. No es necesario desarrollar un algoritmo OCR desde cero. La prioridad es encapsular el motor detrás de una interfaz propia para poder cambiarlo sin afectar la API.

Separar claramente OCR de Document Understanding: OCR convierte imagen→texto; la extracción estructurada convierte texto/layout→campos como NIT, total, fecha o número de factura.

5.4 PDF Generator + API

```

POST /v1/pdf/generate
{
  "template_id": "invoice_v1",
  "data": {
    "customer": "Juan Pérez",
    "total": 250000,
    "items": []
  }
}
  
```

```

→ job_id
→ resultado PDF
  
```

- Generación síncrona solo para archivos pequeños y rápidos.

- Generación asíncrona para lotes.
- Plantillas versionadas.
- Resultado descargable por URL temporal.
- API idempotente para evitar duplicados.
- Webhooks de finalización para integraciones.

6. API

La API debe ser el contrato estable del producto. El dashboard consume la misma capa de negocio.

```

/v1
├── /documents
│   ├── POST /process
│   └── GET /:id
├── /jobs
│   ├── GET /:id
│   └── POST /:id/cancel
├── /csv
│   └── POST /process
├── /pdf
│   └── POST /generate
├── /templates
├── /usage
└── /webhooks
  
```

Usar autenticación con API keys, scopes cuando sea necesario, rate limiting, idempotency keys, validación de payloads y versionado /v1.

7. Sistema de créditos y cobro

La unidad comercial debe abstraer el costo real. Una operación simple no debe costar lo mismo que OCR de 100 páginas.

Operación	Ejemplo de consumo inicial
Generar PDF simple	1 crédito
Comprimir/dividir PDF pequeño	1 crédito
Extraer texto	1-2 créditos
CSV pequeño	1 crédito
CSV grande	5+ créditos según tamaño
OCR	10+ créditos según páginas
Extracción avanzada	precio según complejidad

Estos valores son de diseño inicial, no precios finales. Deben calibrarse con benchmarks de costo real por job.

7.1 Planes iniciales

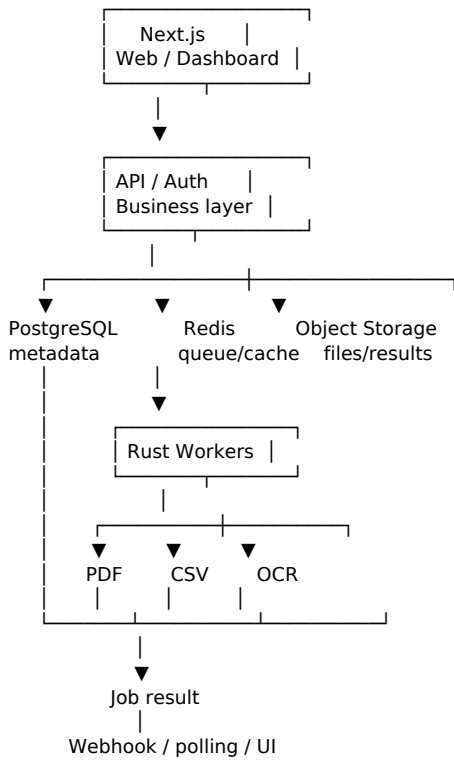
Plan	Precio de referencia	Incluye
Free	\$0	100 créditos, límites bajos, archivos pequeños, API limitada
Starter	\$9/mes	5,000 créditos, 50 MB/archivo, API, lotes
Pro	\$29/mes	30,000 créditos, 250 MB/archivo, mayor velocidad, webhooks
Business	\$99/mes	150,000 créditos, 1 GB/archivo, equipos, prioridad
Enterprise	Personalizado	SLA, límites personalizados, soporte, despliegue/seguridad según contrato

Para Colombia se pueden mostrar precios equivalentes en COP y aceptar los medios de pago disponibles mediante el proveedor elegido. Mantener una moneda base internamente para contabilidad y reporting.

7.2 Billing

- Suscripción mensual/anual.
- Créditos incluidos.
- Compra de créditos extra.
- Prorratio y upgrades/downgrades según proveedor de billing.
- Historial de facturación.
- Alertas de consumo al 50/80/100%.
- Bloqueo o modo degradado cuando se agotan créditos, según plan.

8. Arquitectura técnica recomendada



8.1 Tecnologías

Capa	Tecnología recomendada	Motivo
Frontend	Next.js + TypeScript	SSR/SEO, routing, UI y ecosistema conocido
UI	Tailwind CSS + componentes propios	Velocidad y consistencia visual
API/orquestación	Next.js inicialmente o servicio dedicado según diseño	Reducción de complejidad del MVP
Procesamiento	Rust	CPU, concurrencia, workers y binarios eficientes
Queue	Redis + sistema de jobs	Trabajos asíncronos y reintentos
DB	PostgreSQL	Usuarios, jobs, usage, billing, metadata
Storage	S3-compatible	Archivos grandes y URLs temporales
OCR	Motor open source/self-hosted	Evitar costo por página de proveedor

Capa	Tecnología recomendada	Motivo
Observabilidad	OpenTelemetry + logs/metrics	Trazabilidad de jobs
Tests	Vitest + Playwright/Cypress + tests Rust	Unitarios, integración y E2E
CI/CD	GitHub Actions	Lint, tests, build y deploy

8.2 Modelo de procesamiento

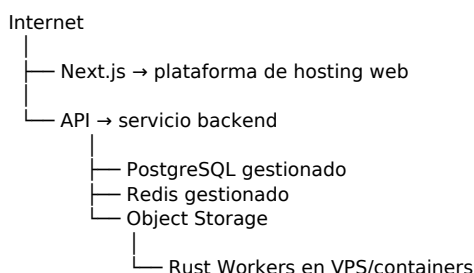
1. Cliente solicita job
2. API valida usuario, plan, tamaño y créditos
3. Se crea Job = QUEUED
4. Archivo entra a storage
5. Se publica mensaje en queue
6. Worker Rust toma job
7. Job = PROCESSING
8. Worker descarga/lee streaming
9. Ejecuta operación
10. Resultado se guarda en storage
11. Metadata + consumo se registra en PostgreSQL
12. Job = COMPLETED / FAILED
13. API/webhook notifica
14. Archivos temporales expiran

9. Seguridad

- Nunca confiar en nombre/extensión MIME enviados por el usuario.
- Validar tamaño, tipo, cantidad de archivos y límites del plan.
- Guardar archivos fuera del filesystem público.
- Usar URLs firmadas/temporales para uploads y downloads.
- Aislar workers y limitar CPU/memoria/tiempo por job.
- Desactivar ejecución de archivos subidos.
- Antivirus/malware scanning cuando el modelo de negocio lo requiera.
- Cifrado en tránsito y, preferiblemente, en reposo.
- API keys almacenadas como hashes cuando sea viable; mostrar el secreto completo una sola vez.
- Rate limiting por usuario/API key/IP.
- Auditoría de acciones administrativas.
- Política clara de retención y eliminación de archivos.

10. Despliegue recomendado

Para el MVP conviene separar web de workers, porque el procesamiento pesado no debe depender de funciones serverless con límites de CPU/tiempo.



Estrategia práctica: Next.js puede desplegarse en una plataforma optimizada para frontend/web; los Rust workers en contenedores sobre un VPS/servicio de containers. A medida que crezca el volumen, aumentar

workers horizontalmente.

10.1 Entornos

development
staging
production

Cada entorno debe tener:

- DB separada
- Redis separado
- Storage/bucket separado
- Secrets separados
- API keys separadas

10.2 CI/CD

- Pull Request → lint + typecheck + unit tests + Rust tests.
- Build de Next.js y binarios/containers Rust.
- Tests de integración contra servicios efímeros cuando sea posible.
- Deploy automático a staging.
- Smoke tests.
- Promoción a producción con aprobación para cambios sensibles.
- Migraciones DB versionadas y reversibles cuando sea posible.

11. Pruebas

Nivel	Qué probar
Unitarias	Parsers, normalizadores CSV, cálculo de créditos, validadores, lógica de plantillas
Integración	Postgres, Redis, storage, queue, API y workers
E2E	Registro → upload → job → resultado → billing/usage
Carga	Muchos jobs simultáneos, archivos grandes, backpressure
Seguridad	Archivos maliciosos, path traversal, MIME spoofing, auth, rate limits
Resiliencia	Worker muerto, Redis caído, timeout, retry, job duplicado
OCR	Sets de documentos reales y métricas de precisión
PDF	Fuentes, tablas, imágenes, páginas múltiples y PDFs corruptos

12. Observabilidad

- Cada job debe tener job_id/correlation_id.
- Métricas: jobs/minuto, éxito, fallo, latencia p50/p95/p99, CPU/RAM, queue depth.
- Consumo: créditos por usuario/operación.
- Errores agrupados por código.
- Alertas: queue creciendo, worker caído, storage fallando, tasa de error elevada.
- Dashboard interno para salud del sistema.

13. Plan de desarrollo 0% → 100%

Fase	Resultado
0 — Descubrimiento	PRD, casos de uso, costos, límites y modelo comercial
1 — Fundación	Repo, CI/CD, arquitectura, auth, DB, storage, UI base
2 — Jobs	Modelo Job, queue, estados, retries, workers Rust
3 — PDF	Upload, extracción, unión/división/compresión y resultados
4 — CSV	Streaming, limpieza, duplicados, validaciones y exportación
5 — PDF API	Templates, generación, API keys, documentación
6 — OCR	Pipeline local, benchmarks, detección digital/escaneado
7 — Billing	Planes, créditos, usage, pagos, límites
8 — Seguridad/QA	Carga, E2E, hardening, observabilidad, backups
9 — Beta	10–30 usuarios, feedback, costos reales, ajustes
10 — Producción	Landing, docs, pricing, soporte, monitoring, backups
11 — Escala	Workers horizontales, caching, optimización y enterprise

13.1 MVP exacto recomendado

Para no convertir el primer lanzamiento en un monstruo, el MVP debería tener:

- Auth + organizaciones simples.
- Dashboard.
- Upload de PDF/CSV.
- PDF: extraer texto, unir, dividir y comprimir.
- CSV: duplicados, normalización básica, dividir/combinar.
- PDF Generator con plantillas simples.
- API keys + 3 endpoints.
- Jobs asíncronos con Rust worker.
- Storage + PostgreSQL + Redis.
- Usage + créditos.
- Un plan Free y uno Pro inicialmente.
- Logs, retries, límites y eliminación automática.

OCR puede entrar inmediatamente después del MVP, cuando el pipeline de jobs esté probado. Así no se mezcla el riesgo de infraestructura con el riesgo de precisión OCR.

14. Modelo de datos inicial

```

users
organizations
organization_members
subscriptions
plans
api_keys
files
jobs
job_inputs
job_outputs
operations
usage_events
credit_balances
credit_transactions

```

pdf_templates
pdf_template_versions
webhook_endpoints
webhook_deliveries
audit_logs

El modelo exacto debe ser refinado por Cursor antes de implementar. La separación entre metadata y archivos binarios es obligatoria: PostgreSQL no debería almacenar PDFs grandes como blobs por defecto.

15. Ejemplos de API

POST /v1/files/presign
→ URL temporal para upload

```
POST /v1/jobs
{
  "operation": "csv.clean",
  "file_id": "file_123",
  "options": {
    "remove_duplicates": true
  }
}
```

```
→ {
  "job_id": "job_456",
  "status": "queued"
}
```

```
GET /v1/jobs/job_456
→ {
  "status": "completed",
  "progress": 100,
  "result": {...}
}
```

```
POST /v1/pdf/generate
{
  "template_id": "invoice_v1",
  "data": {...},
  "idempotency_key": "..."
}
```

POST /v1/webhooks/test
→ prueba de entrega

16. Economía unitaria

Antes de fijar precios definitivos, medir por operación:

- CPU-segundos.
- RAM/GB-hora si aplica.
- GB transferidos.
- GB-mes de storage.
- Páginas OCR.
- Tiempo de worker.
- Costo de servicios externos (si algún día se añaden).
- Costo promedio por job y p95.

```
Costo_job =
  compute
+ storage temporal
+ bandwidth
+ OCR externo (si existe)
+ otros costos variables
```

Margen bruto objetivo:
precio_consumo - costo_job >> 0

Recomendación: reservar margen amplio para jobs pesados y no vender OCR ilimitado barato hasta tener benchmarks.

17. Página de ventas

HERO

Procesa documentos y datos a escala.

PDF • CSV • OCR • API

Sube archivos o integra nuestra API para extraer, transformar y generar documentos.

[Probar gratis] [Ver API]

SECCIÓN 2

¿Qué puedes procesar?

[PDF] [CSV] [OCR] [PDF API]

SECCIÓN 3

Antes → Después

10,000 facturas

↓

datos estructurados

CSV / JSON / API

SECCIÓN 4

¿Cómo funciona?

Upload → Process → Result

SECCIÓN 5

API

POST /v1/jobs

SECCIÓN 6

Precios

Free / Starter / Pro / Business

SECCIÓN 7

Seguridad / Retención

CTA FINAL

Empieza gratis

El marketing debe vender resultados (“procesa 10.000 documentos”) y no la tecnología (“Rust, Redis, workers”). La tecnología es un argumento secundario para la página técnica.

18. Instrucciones para Cursor

El siguiente bloque puede copiarse como contexto inicial para que Cursor produzca el plan técnico.

Actúa como Staff/Principal Engineer y Product Architect.

Quiero diseñar y construir una plataforma SaaS API-first para procesamiento masivo de documentos y datos.

Stack objetivo:

- Next.js + TypeScript para web/dashboard
- Rust para workers y procesamiento intensivo
- PostgreSQL
- Redis/queue
- S3-compatible object storage
- Docker
- GitHub Actions
- Testing con Vitest + Playwright/Cypress y tests nativos de Rust

Módulos:

1. PDF/documentos
2. CSV/data processing
3. PDF generation
4. OCR self-hosted
5. API keys/API v1
6. Jobs, queue, retries
7. Usage/credits

- 8. Billing
- 9. Dashboard
- 10. Observability/security

IMPORTANTE:

- No implementes todo de una vez.
- Primero analiza requisitos, riesgos, costos y arquitectura.
- Divide el proyecto en fases entregables.
- Para cada fase define: objetivo, tareas, dependencias, archivos/capas afectadas, migraciones, endpoints, tests, criterios de aceptación y estrategia de deploy.
- Prioriza un MVP pequeño y comercializable.
- Mantén PDF/CSV/OCR como operaciones desacopladas mediante una interfaz de job/operation para poder agregar nuevos procesadores.
- Diseña el sistema para ejecutar workers Rust horizontalmente.
- Los archivos deben ir a object storage; PostgreSQL guarda metadata.
- Usa URLs firmadas para upload/download.
- Los jobs pesados son asíncronos.
- Incluye idempotencia, retries, timeouts, dead-letter/recovery strategy.
- Diseña créditos con consumo variable según operación.
- No entrenes OCR desde cero inicialmente; evalúa motores open source self-hosted y encapsúlalos detrás de una interfaz.
- Incluye benchmarks de OCR y procesamiento.
- Incluye seguridad de archivos y aislamiento de workers.
- Incluye observabilidad y trazabilidad por job_id.
- Genera un plan de despliegue development/staging/production.
- Propón proveedores de infraestructura y una estimación de costo por etapa.
- Identifica qué debe mantenerse en Next.js y qué debe vivir en Rust.
- No agregues microservicios innecesarios al MVP.
- Antes de escribir grandes cantidades de código, presenta arquitectura, ADRs, modelo de datos, contratos API y roadmap.

Entrega primero:

- A) arquitectura general
- B) diagrama de componentes
- C) modelo de datos
- D) contrato API v1
- E) diseño de jobs/queue/workers
- F) estrategia OCR
- G) estrategia de storage
- H) billing/credits
- I) testing
- J) observabilidad
- K) seguridad
- L) despliegue
- M) roadmap 0-100%
- N) backlog priorizado
- O) riesgos y decisiones pendientes.

Cursor debe convertir este documento en un plan técnico concreto, no asumir que todas las decisiones están cerradas. Las decisiones de proveedor, OCR y pricing deben validarse con benchmarks y costos reales.

19. Criterios de “100% listo”

- Landing + pricing + docs publicados.
- Registro, login y organizaciones funcionando.
- Upload seguro a storage.
- Jobs asíncronos con estados y retries.
- PDF y CSV MVP funcionando.
- PDF API documentada.
- OCR self-hosted probado con dataset representativo.
- Credits/usage/billing funcionando.
- API keys + rate limits + idempotencia.
- Backups y recuperación probados.

- Observabilidad y alertas.
- Suite de tests automatizados.
- Prueba de carga y límites.
- Política de retención/borrado.
- Beta con usuarios reales.
- Costos por operación conocidos.
- Planes ajustados al margen.
- Deploy reproducible y rollback documentado.

20. Recomendación final

Construir primero el motor de jobs + storage + usage y encima colocar PDF/CSV. El motor será el activo reutilizable. OCR entra después como una operación adicional. La generación de PDF debe compartir el mismo sistema de jobs y credits.

La visión final no es “una página para convertir PDFs”, sino una plataforma de procesamiento de documentos y datos con dos interfaces: dashboard para humanos y API para software.

La ventaja técnica buscada con Rust es eficiencia y capacidad de procesamiento controlada por ti, especialmente en workers y operaciones intensivas. La ventaja comercial será la simplicidad: el cliente sube archivos o llama una API y obtiene resultados.

Siguiente paso recomendado: entregar este documento a Cursor y pedirle que produzca la arquitectura, ADRs, modelo de datos, contrato API y roadmap del MVP antes de comenzar a programar.